

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

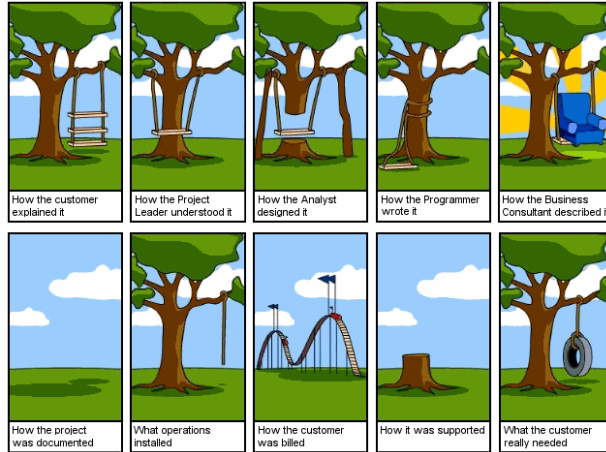
- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szekvenciális vezérlések
 - 10 Python iterációk**
 - Csúszólagos számítás
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Miből áll a programozás folyamata?

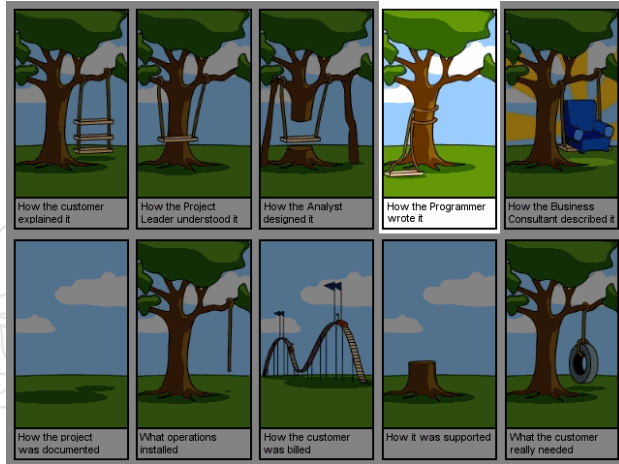
Milyen lépései vannak?



<https://www.projectcartoon.com/>

Miből áll a programozás folyamata?

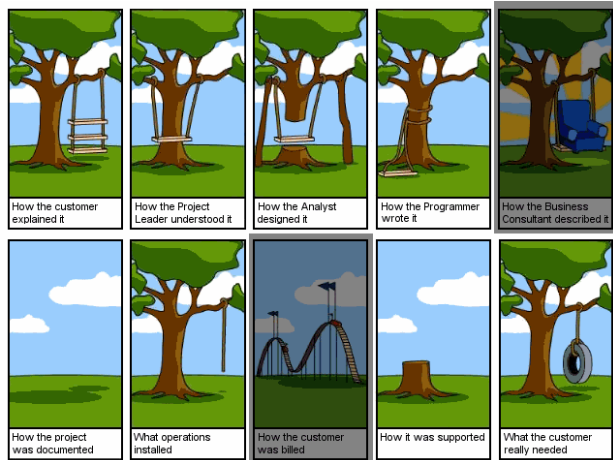
Milyen lépései vannak?



<https://www.projectcartoon.com/>

Miből áll a programozás folyamata?

Milyen lépései vannak?



<https://www.projectcartoon.com/>

A programozás fázisai

Szoftverfejlesztési modellek

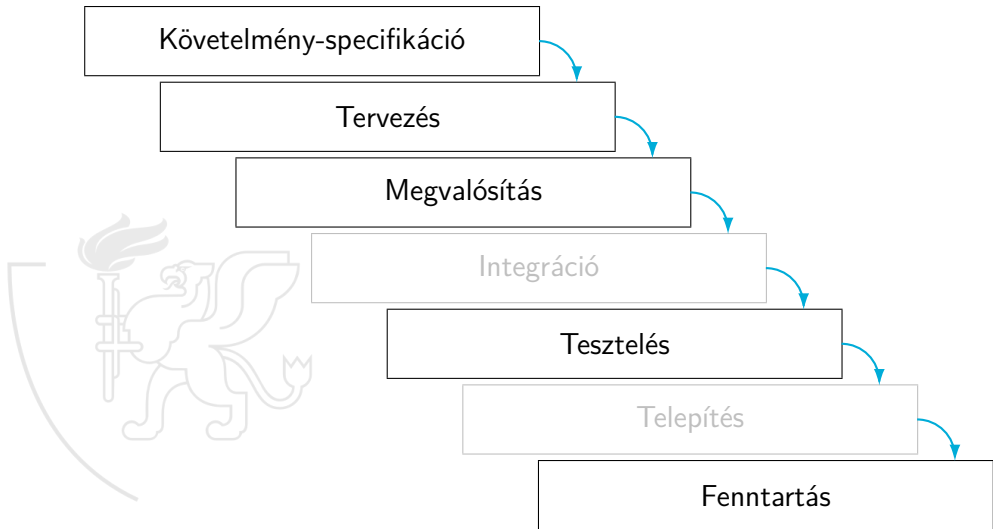
- A számítógépes problémamegoldás (a programozás) egymástól jól elkülöníthető fázisokból épül fel, amelyek sajátos kölcsönhatásban vannak egymással; ezt a kapcsolatot fejezi ki a *szoftverfejlesztési modell* (Software Development Life Cycle – SDLC).
- A világon többféle modell létezik, ezek közül nincs jó vagy rossz, csak adott feladathoz jobban vagy kevésbé alkalmas.
 - Vízesés-modell
 - V-modell
 - spirál-modell
 - O-modell
 - extrém programozás
 - ...

A programozás fázisai

Vízesés modell

- Az egyik első és legegyszerűbb a vízesés (waterfall) modell.
 - Követelmény-specifikáció (Problémafelvetés, Specifikáció)
 - Tervezés (Algoritmustervezés)
 - Megvalósítás
 - Integráció
 - Tesztelés (Verifikáció, Validáció)
 - Telepítés
 - Fenntartás (Végrehajtás, Fenntartás)
- A modellben a fázisokat sorban egymás után hajtjuk végre.

A vízesés modell



A vízesés modell fázisai

Követelmény-specifikáció

- A (későbbi) felhasználó a saját szakterületének megfelelő nyelven meg tudja fogalmazni a megoldandó problémát (*problémafelvetés*), ez viszont az algoritmus elkészítéséhez nem elég.
- Az algoritmushoz pontosan meg kell határozni, hogy milyen feltételt elégítenek ki a probléma bemenő adatai, és hogy adott bemenő adatok esetén milyen feltételt kell kielégíteni a kiszámított adatoknak (*specifikáció*).
- A specifikáció tehát egy (B, K) bemeneti-kimeneti feltétel pár: akkor fogadjuk el az algoritmust a probléma megoldásának,
 - ha valahányszor a B feltétel teljesül a bemenő adatokra,
 - mindannyiszor a K feltétel teljesül a kiszámított adatokra.
- A specifikáció meghatározásánál szinte teljes mértékben szabad kezünk van, feltéve, hogy a „mi” nem csak a programozót, hanem a program elkészítésében érdekelt összes felet jelenti.

- Itt történik a specifikációt kielégítő algoritmus létrehozása (*algoritmustervezés*).
- A felülről lefelé haladó (top-down) módszer lényege:
 - A kiindulási P problémát $P_1, \dots, P_n$ részproblémákra bontjuk.
 - Minden P_i problémának megadjuk valamely M_i megoldását.
 - Az M_i műveleteket alkalmas módon összetéve a P problémát megoldó algoritmushoz jutunk.
 - Ezt (rekurzívan) addig folytatjuk, amíg a választott programozási nyelv elemi műveletével közvetlenül meg nem adható problémákhoz jutunk.
- Az algoritmus meghatározásában mindaddig szabad kezünk van, amíg az a megadott specifikációnak megfelelően működik (figyelembe véve, hogy a specifikációban **nem** szereplő funkciókat a programnak **nem** kell és **nem** is szabad ellátnia).

A vízesés modell fázisai

Megvalósítás

- Megvalósításon az algoritmustervezés során kifejlesztett algoritmusnak egy adott programozási nyelven történő leírását értjük (*kódolás*).
- A lépés eredménye a program.
- A megvalósítás során szigorúan követnünk kell a tervezés során megalkotott algoritmust; a lehetséges variánsok, alternatívák csupán technikai jellegűek.



A vízesés modell fázisai

Integráció

- Amennyiben az általunk megoldott probléma egy nagyobb, összetettebb probléma része, úgy a programunknak is szükségképpen illeszkednie kell egy a nagyobb probléma egyéb komponenseinek megvalósításához.
- Ilyen esetekben ez a lépés szolgálja a saját kódunk nagyobb programrendszerbe történő beillesztését, és a beillesztés során esetleg felvetődő problémák megoldását.



A vízésés modell fázisai

Tesztelés

- A tesztelésnek része a
 - *verifikáció*, minek során azt ellenőrizzük, hogy a kifejlesztett szoftver megfelel-e a követelményeknek; és a
 - *validáció*, amikor azt vizsgáljuk, hogy (a követelményeknek megfelelő, verifikált) szoftver megfelelően használható-e arra a célra, amire tervezték (vagyis jók voltak-e a követelmények).
- A tesztelés jellemzői
 - A tesztelés nem alkalmas program helyességének bizonyítására, pusztán segít felmérni a szoftver állapotát és ezáltal hozzájárul a minőség növeléséhez.
 - A tesztelésnek nagyon sok formája van.

A vízesés modell fázisai

Telepítés

- Amennyiben szükséges a programot telepíteni kell, azaz el kell helyezni és működőképessé kell tenni a felhasználó által kívánt környezetben.



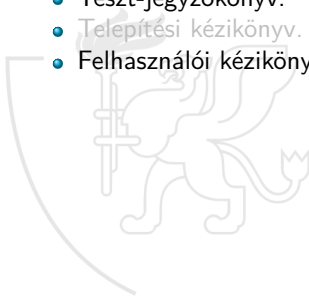
A vízesés modell fázisai

Fenntartás

- A programkészítés végső célja az, hogy a megoldandó probléma konkrét bemenő adataira végrehajtsuk a kifejlesztett algoritmust.
- A program fenntartásán a végrehajtás során felmerült problémák megoldását értjük, ide tartozik például:
 - a használat során felmerülő hibák kijavítása, vagy
 - kisebb/nagyobb módosítások elvégzése.



- Nagyon fontos, hogy a teljes folyamat dokumentálva legyen, és ne csak ott, ahol ez nyilvánvalóan szükséges.
 - A specifikáció önmagában dokumentum.
 - A tervek valamilyen formájú leírása.
 - Megjegyzések a kódban.
 - Integráció során történt módosítások leírása.
 - Teszt-jegyzőkönyv.
 - Telepítési kézikönyv.
 - Felhasználói kézikönyv, hibajavítások, módosítások rögzítése.



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A programok fő komponensei

- Minden programnak két fő komponense van:
 - az adatok, és
 - az adatokon végzett műveletek.
- Az adatok is és a műveletek is lehetnek elemiek és összetettek.
- Adatokból adat-összetételi, műveletekből műveletképzési szabályokkal összetett adatok illetve műveletek gyárthatók.



Algoritmus vezérlése

Az az előírás, amely az algoritmus minden lépésére (részműveletére) kijelöli, hogy a lépés végrehajtása után melyik lépés végrehajtásával folytatódjon (esetleg fejeződjön be) az algoritmus végrehajtása.

- Az algoritmusnak, mint műveletnek a vezérlés a legfontosabb komponense.



Vezérlési módok

- A vezérlési mód azt fejezi ki, hogy egyszerűbb műveletekből hogyan építünk fel összetett műveletet és ennek milyen lesz a vezérlése.
- Négy fő vezérlési módot különböztetünk meg:

Szekvenciális Véges sok adott művelet rögzített sorrendben egymás után történő végrehajtása.

Szelekciós Véges sok rögzített művelet közül adott feltétel alapján valamelyik végrehajtása.

Ismétléses Adott művelet adott feltétel szerinti ismételt végrehajtása.

Eljárás Adott művelet alkalmazása adott argumentumokra, ami az argumentumok értékének pontosan meghatározott változását eredményezi.

- A Python művelet-összetételi konstrukciói többé-kevésbé besorolhatók a négy vezérlési mód valamelyikébe.
- A vezérlési módok ugyanakkor nyelvek feletti fogalmak.
- A imperatív (algoritmikus) programozási nyelvekben ezek a vezérlési szerkezetek (közvetlenül vagy közvetve) megvalósíthatók.



Tartalomjegyzék

- 
- 1 Bemutatók**
- Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
- 2 Python alapok**
- Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
- 3 Alapfogalmak**
- Alapfogalmak
 - Alapvető programelemek fogalmak
- 4 Python feltételes ciklusok**
- Röppálya számítás
 - Röppálya szimuláció
- 5 Szoftverfejlesztési életciklus modellek**
- A programozás fázisai
- 6 Python sztringek**
- Python sztringek
 - Basename meghatározása
- 7 Vezérlési szerkezetek**
- Bevezetés
 - Szekvenciális vezérlés
 - Függvények
- 8 Python listák**
- Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
- 9 Vezérlési szerkezetek**
- Szelekciós vezérlések
- 10 Python iterációk**
- Csúszóátlag számítása
- 11 Vezérlési szerkezetek**
- Ismétléses vezérlések
- 12 Python halmazok, comprehension**
- Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
- 13 Vezérlési szerkezetek**
- Ismétléses vezérlések
- 14 Python dictionary, függvény mint típus**
- Python dict adattípus
 - Python comprehension
 - Python function adattípus
- 15 Vezérlési szerkezetek**
- Eljárásvezérlés
- 16 Python fájlkezelés**
- Fájlkezelés
- 17 Vezérlési szerkezetek**
- Eljárásvezérlés
- 18 Python formátált IO**
- Python formátált IO
- 19 Adatkezelés, adattárolás**
- Az adatkezelés szintjei
 - Az adattárolás jellemzői
- 20 Python típusok**
- Python numerikus típusok
- 21 Absztrakt adattípusok**
- Egyszerű adattípusok
 - Összetett adattípusok
- 22 Python típusok**
- További Python típusok
 - Python function adattípus
- 23 Absztrakt adattípusok**
- Összetett adattípusok
- 24 Tesztelés**
- Tesztelés
 - Tesztelés Python-ban
 - A pytest
- 25 Vizualizáció és egyéb modulok**
- Fájlkezelő modulok
 - Vizualizáció
- 26 More Python**
- A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Értékadó utasítás

- Az értékadás jele az =
- Python-ban egyszerre több változónak is értéket lehet adni.

```
>>> a, b, c = 1526, 'Mohács', ('II. Lajos', 1506, 1526)
>>> a
1526
>>> c
('II. Lajos', 1506, 1526)
```

- Sőt, tuple és list típusok esetén az ilyen értékadás „okosan” működik.

```
>>> x, y, z = c
>>> x
'II. Lajos'
>>> y
1506
```

- Most mélyebb magyarázat nélkül átnézzük, hogy a Python nyelv milyen lehetőségeket ad összetett utasítások képzésére, illetve megadjuk ezen konstrukciók szintaxisát.



Utasítások sorozata

- Ha az utasításokat adott sorrendben kell végrehajtani, akkor egyszerűen *azonos indentálással* (behúzással) egymást követő sorokban (teljesen üres sort megengedve közöttük) leírjuk őket
- Számoljuk ki, melyik órában (o), percben (p) melyik másodperc (m) lesz a nap 54321. másodperce.

```
e = 54321
o = e / 3600
e = e % 3600
p = e / 60
m = e % 60
```

Függvény

A függvény a matematikai értelemben vett függvény általánosítása (bővítése), gyakorlatilag egy (rész)algoritmus megvalósítása.

- Egy-egy függvény valamilyen bemenő adatok alapján kiszámol egy értéket, mint ahogyan azt a matematikában már megszokhattuk.
- Vannak olyan függvények, amiknek valamilyen jól definiált mellékhatása is van a visszatérési érték kiszámítása mellett.
- Adott esetben az is előfordulhat, hogy egy függvénynek csak a mellékhatása fontos, és (matematikai értelemben) nem ad vissza semmilyen értéket, pl. a `print(...)`. (Az ilyen függvényeket nevezhetjük eljárásnak.)
- Python-ban:
 - Egy függvény látszólag több értéket is visszaadhat.
 - A „nem ad vissza értéket” a `None` érték visszaadásával valósul meg.

- Python nyelven függvényeket lehet definiálni és meghívni.
 - Definíciónál meg kell adnunk, hogy a függvényünk milyen paraméterekből milyen értéket hogyan fog kiszámolni, vagyis:
 - Meg kell adni a *paraméterek* listáját, és
 - meg kell adni az algoritmust.
 - A függvényhívásnál pedig konkrét *argumentumokra* alkalmazzuk a függvényt, és a kiszámított értéket felhasználhatjuk további számolásainkhoz.



Függvény

Definíció és használat [2/2]

- Függvény definíciót a `def` kulcsszó segítségével készíthetünk:

```
def f(a, b):  
    return a + b
```

- Nagyon fontos a „szépen formázottság”, vagyis a *függvény törzsének* (az algoritmusát leíró utasításoknak) a kódban egy behúzással beljebb kell kezdődniük, mint ahol a függvény definíciója kezdődik! Ez (az *indentálás*) Python-ban nem csak egy ajánlás, hanem kötelező, a nyelv szintaktikájának szerves része!
- Függvényhíváskor a függvény nevét zárójelben a hívás argumentumai követik:

```
c = f(9, 3 * 11)
```

- Python-ban a paraméterezés többféle lehet; a legegyszerűbb sorrendi argumentumoknál a definíciónál felsorolt paraméterek sorrendben kapják meg a hívásból megadott argumentumokat.

A return utasítás

- Minden függvényben lehet egy vagy több return utasítás.
- Ha a függvényben egy ilyen utasítást hajtunk végre, akkor a függvény értékének kiszámítása befejeződik. A hívás helyén a függvényhívás kifejezés a return által kiszámított értéket veszi fel.
- Ha egy függvényben nincs return utasítás, vagy a függvény nem ennek végrehajtásával ér véget, az olyan, mintha egy return None-t hajtottunk volna végre.

```
def f(a, b):  
    return a + b
```

- Írjunk egy programot, ami óra, perc, másodperc alapján egy függvény segítségével kiszámolja az éjfél óta eltelt másodperceket.

```
def masodpercek(ora, perc, masodperc):  
    return 3600 * ora + 60 * perc + masodperc
```

```
mp1 = masodpercek(12, 5, 7)  
mp2 = masodpercek(6, 45, 0)
```

Az if utasítás

- Ha valamilyen feltétel alapján egyik vagy másik utasítást akarjuk végrehajtani.
 - Végre kell-e hajtani valamit?
 - Számoljuk ki f abszolút értékét!

```
if f < 0:  
    f = -f
```

- Két utasítás közül az egyiket hajtsuk végre!
 - a és b közül melyik a kisebb érték?

```
if a <= b:  
    k = a  
else:  
    k = b
```

A while utasítás

- Ha valamilyen műveletet mindaddig végre kell hajtani, amíg egy feltétel igaz a `while` utasítás használható.
 - A művelet végrehajtása nem szükséges a feltétel kiértékeléséhez.
 - A feltétel ellenőrzése a művelet előtt történik, így ha a feltétel kezdetben hamis volt, a műveletet egyszer sem hajtjuk végre.

```
while sum_of_hand < 19:  
    sum_of_hand += draw_new_card()
```



A for utasítás

- Ha valamilyen műveletet több előre megadott értékre is végre kell hajtani, akkor a for utasítás használható.
 - Hajtsunk végre egy műveletet adott számú alkalommal.
 - „És most büntetésből százszor leírod a nevedet!”

```
>>> for i in range(100):  
...     print("neved")
```



Az else „utasítás”

- Nem önálló utasítás, az if, while és for utasítások része (bővítése).
- Mit csinálunk, ha egy utasítás feltétele (már) nem teljesül?

- Egyszerű feltétel

```
if feltetel():  
    igaz()  
else:  
    hamis()
```

- Kezdőfeltételes ciklus

```
while feltetel():  
    igaz()  
else:  
    hamis lett()
```

- Elemvezérelt ciklus

```
for v in iterable():  
    van_meg_elem()  
else:  
    nincs_mar_elem()
```

- Ciklusok esetén az else ág nem hajtódik végre, ha a ciklusból break segítségével lépünk ki.

A break és continue utasítások

- A Python nyelvben a ciklusmag folyamatos végrehajtásának megszakítására két utasítás használható:
 - break** Megszakítja a ciklust, a program végrehajtása a ciklus utáni első utasítással folytatódik. A ciklushoz tartozó else ág **nem** hajtódik végre.
 - continue** Megszakítja a ciklusmag aktuális lefutását, a vezérlés a ciklus feltételének kiértékelésével (`while`) illetve a soron következő elem feldolgozásával (`for`) folytatódik.



A try-except utasítás

- Ha valamilyen lehetséges hiba következményeit szeretnénk kezelni.
 - Mit csináljunk, ha futási hiba lép fel egy utasítás végrehajtása során?
 - Ha a felhasználó nem számot ad meg, akkor nem számolunk.
 - De ha nullát, akkor is baj van.

```
try:
    v = int(input('Kérek egy egész számot: '))
    if v % 2 == 0:
        print('A szám páros, a reciproka %f!' % (1 / v))
    else:
        print('A szám páratlan!')
except:
    print('Valami hiba történt!')
```

A with utasítás

- Ha valamilyen „erőforrás” használata megfelelő előkészítést és lezárást is igényel a használat előtt/után, akkor a with utasítás (is) használható.
 - Az as használható arra, hogy az erőforráshoz egy azonosítót rendeljünk.
 - Az as rész elhagyható, ha az erőforrás használata transzparens
 - Ha egy fájlt akarunk használni, a with utasítás kézenfekvő megoldás.

```
with open('output.txt', 'w') as f:  
    f.write('Helló_Világ!\n')
```

Egyszerű ki- és bevétel

- Az egyszerűbb programok be- és kiviteli utasításokkal kommunikálnak a környezetükkel.
- Ahhoz, hogy működő példát mutassunk be, szükségünk van ezekre az utasításokra is.
- A Python nyelv tartalmaz néhány alap ki- és beviteli utasítást illetve függvényt.



Egyszerű kivitel

`print()`

- `print(...)`:
 - Python „beépített” (*built-in*) függvény
 - Alapvetően a standard kimenetet (`sys.stdout`) használja, ez alapesetben a terminál / képernyő
 - **0**: Kiírja az argumentumként kapott sztringet
 - **1**: Kiírja az argumentum értékét, sztringgé alakítva
 - **2**: Kiírja az argumentumok értékét egy (sorvége jellel lezárt) sorban, szóközzel elválasztva

```
>>> print()

>>> print('Hello!')
Hello!

>>> print(2024)
2024

>>> y=2024
>>> h="Hi!"
>>> y
2024

>>> print(h, y)
Hi! 2024
```

Egyszerű bevitel

`input()`

- `input(...)`:
 - Python „beépített” (*built-in*) függvény
 - Alapvetően a standard bemenetet (`sys.stdin`) használja, ez alapesetben a billentyűzet
 - **0**: Visszaadja a beolvasott teljes sort (sorvége jel nélkül) sztringként
 - **1**: Megadhatunk egy sztringet, amit a beolvasás előtt kiír („prompt”)

```
>>> input()
hello
'hello'
>>> input('Kérdés?_')
Kérdés? van
'van'
>>> n = input('n_értéke:_')
n értéke: 6
>>> print(n)
6
>>> n
'6'
```